Atividade 07 — Delivery Tracker: Persistência com Prisma (SQLite)

Disciplina: Programação Web II — Backend (Node/Express) · IFAL/Maceió **Capítulo:** 5 — Banco de Dados e ORM (Prisma) **Modalidade:** Individual · **Peso:** Exercício Cap. 5 (avaliado) **Correção:** automática (autograder de conformidade) + persistência + arquitetura

> **Pré-requisito:** Exercício Cap. 4 concluído (Delivery Tracker em camadas, com Repository/DI e persistência **em memória**). Você continua **no mesmo repositório** `pweb2-delivery-<matricula>`.

Contexto

Até aqui o Delivery Tracker guarda tudo **em memória** — ao reiniciar o servidor, os dados somem. Agora a empresa precisa de **persistência real**: as entregas e motoristas devem sobreviver a reinícios, quedas e deploys. Você vai trocar o repositório em memória por uma implementação com **Prisma ORM** sobre **SQLite**, **sem mudar o comportamento da API**.

O ponto central desta atividade é a **prova do Repository Pattern**: você troca **só a camada de persistência** — `services` e `controllers` continuam **idênticos**.

Objetivos de aprendizagem

- Modelar entidades com **Prisma** (schema) e gerar o banco com **migrations versionadas**.
- Implementar **repositories com Prisma** que respeitam o **mesmo contrato** do Cap. 4.
- Usar um **singleton** do Prisma Client e configurar o banco por **variável de ambiente**.
- Compreender a diferença prática entre persistência **em memória** e **em banco**.

O que muda (e o que NÃO muda)

Muda: - Novo `prisma/schema.prisma` com os modelos **Entrega**, **Motorista** e o **histórico de eventos** (ex.: um modelo `Evento` relacionado a `Entrega`, ou um campo apropriado) — de modo que `GET /api/entregas/:id` continue devolvendo `historico` como um array. - Novos repositories Prisma (ex.: `*.repository.prisma.js`) implementando o **mesmo contrato** (`listarTodos`, `buscarPorId`, `criar`, `atualizar`, `buscarPorCpf`…). - Um **singleton** do Prisma Client (ex.: `src/config/database.js`). - O banco configurado por `DATABASE_URL` (SQLite). - A composição de dependências passa a injetar os repositories **Prisma** (troca de **uma linha** por repository no *composition root*).

NÃO muda: - As **rotas** e todo o **contrato da API** (mesmos métodos, caminhos, status e formato de erro `{erro}`). - Os **services** (regras de negócio) e os **controllers**.

Requisitos

- **RF-01 — Schema Prisma:** `prisma/schema.prisma` com `datasource` **SQLite** e os modelos Entrega, Motorista e o histórico. CPF do motorista **único** no schema.

- **RF-02 — Migrations versionadas:** criadas com `npx prisma migrate dev` e **commitadas** (`prisma/migrations/`).
- **RF-03 — Repositories Prisma:** implementam o mesmo contrato dos repositories do Cap. 4; nenhum acesso ao Prisma Client fora da camada de repository.
- **RF-04 — Persistência real:** os dados **sobrevivem ao reinício** do servidor (é o critério central).
- **RF-05 — Mesmo comportamento:** todas as regras do Cap. 4 continuam valendo (transições de status, duplicidade → 409, origem≠destino → 400, atribuição → 422, etc.).

Convenções (inalteradas desde o Cap. 4)

- Base `/api` · JSON · erro `{ "erro": "..." }` · `GET /api/health` → `{ "status": "ok" }`.
- Execução: `npm start` sobe o servidor, respeita `process.env.PORT`, branch `main`.
- `DATABASE_URL="file:./dev.db"` no `.env` (forneça `.env.example`).

Como rodar

```
npm install
cp .env.example .env            # DATABASE_URL="file:./dev.db"
npx prisma migrate dev          # cria o banco e aplica as migrations
npm start
```

> Para inspecionar os dados: `npx prisma studio`.

Correção automática (autograder)

Como o **contrato da API não mudou**, o mesmo **autograder de conformidade** do Cap. 4 continua valendo (os mesmos 122 pontos de comportamento):

```
npm start
BASE_URL=http://localhost:3000 node autograder/check.mjs   # meta: 122/122
```

Além disso, a **persistência (RF-04)** é verificada assim: cria-se um recurso, **reinicia-se o servidor** e confirma-se que o recurso **ainda existe**. Garanta que, após `npx prisma migrate deploy && npm start`, os dados criados antes continuem lá.

A cada `push`, o **GitHub Actions** roda a correção (lembre de rodar `prisma migrate deploy` / `generate` no workflow, se necessário).

Entregável

- O **mesmo repositório** `pweb2-delivery-<matricula>` (branch `main`), agora com Prisma + SQLite.
- `prisma/schema.prisma` e **`prisma/migrations/` versionadas** (o arquivo `dev.db` e `node_modules/` ficam **fora** do git).
- `README.md` atualizado (como rodar as migrations e subir).

Critérios (transparentes)

Critério	Peso
Autograder — conformidade das rotas (comportamento inalterado)	45%
Persistência real (dados sobrevivem a reinício)	20%
Repositories Prisma respeitando o contrato (services/controllers intactos)	20%
Schema + **migrations versionadas** no git	10%
README atualizado	5%